

Laboratorio 5 · Mexi Banco con Docker Compose

Alumna: Amy Veraza Versión: v05.1 Fecha: 21 de septiembre de 2026

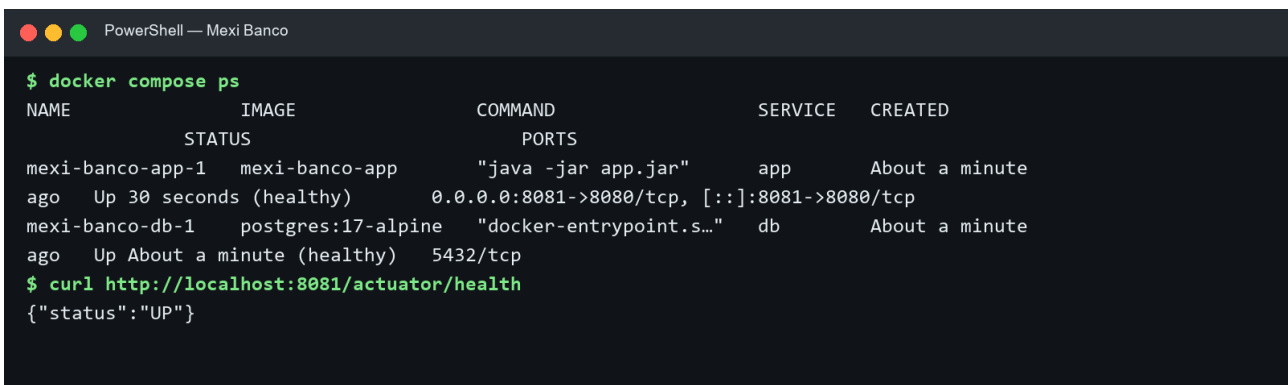
Fuente de evidencia: ejecución automatizada de los mismos endpoints de la colección de Postman; las respuestas completas y los estados de Compose quedaron en resultados.json. El puerto 8080 del equipo estaba ocupado, por lo que publiqué el 8080 del contenedor en localhost:8081.

1. Clonación y arranque

Cloné la etiqueta v05.1 del repositorio oficial. Al construir, el archivo mvnw tenía saltos CRLF, que Alpine no podía ejecutar; cambié a LF solo en la copia local. La imagen se construyó en dos etapas y Compose levantó PostgreSQL y la aplicación. La base llegó a healthy antes de arrancar la aplicación.

```
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE  CREATED
STATUS              PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app      About a minute
ago Up 30 seconds (healthy)  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db        About a minute
ago Up About a minute (healthy)  5432/tcp
```

El endpoint GET /actuador/health devolvió HTTP 200 y estado UP.



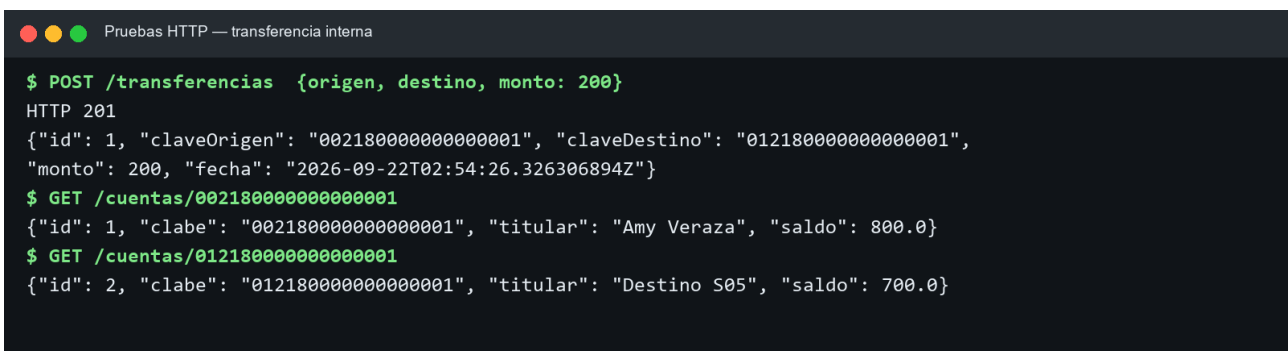
```
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE  CREATED
STATUS              PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app      About a minute
ago Up 30 seconds (healthy)  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db        About a minute
ago Up About a minute (healthy)  5432/tcp
$ curl http://localhost:8081/actuador/health
{"status":"UP"}
```

Captura 1. Compose muestra app y db en estado healthy.

2. Cuentas y transferencia interna

Abrí la cuenta origen con 1000 y la destino con 500. Una transferencia de 200 dejó 800 en origen y 700 en destino. La suma permaneció en 1500: el cargo y el abono cuadran.

```
POST /transferencias → HTTP 201
{
  "id": 1,
  "claveOrigen": "002180000000000001",
  "claveDestino": "012180000000000001",
  "monto": 200,
  "fecha": "2026-09-22T02:54:26.326306894Z"
}
GET origen → saldo 800
GET destino → saldo 700
```



```
$ POST /transferencias {origen, destino, monto: 200}
HTTP 201
{"id": 1, "claveOrigen": "002180000000000001", "claveDestino": "012180000000000001",
"monto": 200, "fecha": "2026-09-22T02:54:26.326306894Z"}
$ GET /cuentas/002180000000000001
{"id": 1, "clabe": "002180000000000001", "titular": "Amy Veraza", "saldo": 800.0}
$ GET /cuentas/012180000000000001
{"id": 2, "clabe": "012180000000000001", "titular": "Destino S05", "saldo": 700.0}
```

3. SPEI e idempotencia

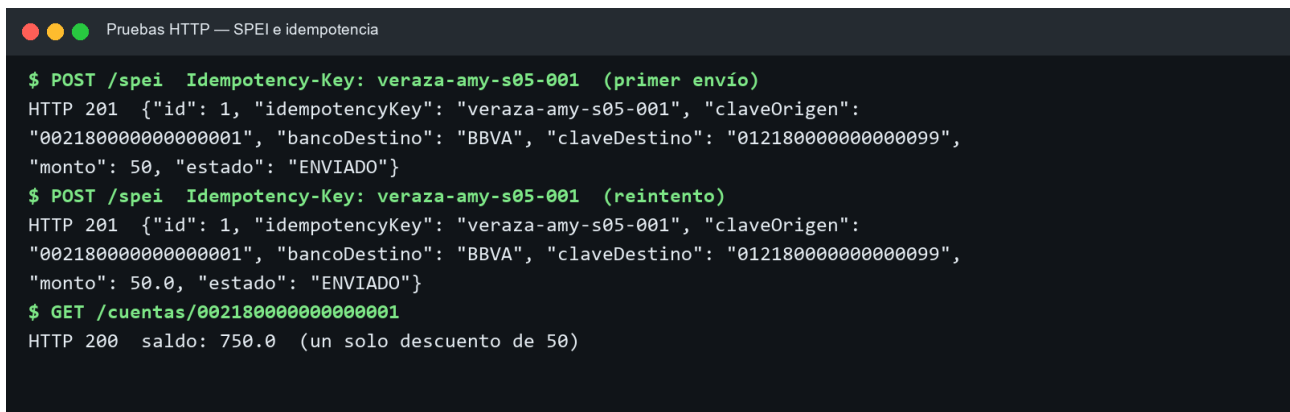
Envíe dos veces el mismo SPEI de 50 con Idempotency-Key: veraza-amy-s05-001. Ambas respuestas refieren al mismo identificador y estado ENVIADO; el saldo final fue 750. El estado de cuenta registró un solo movimiento SPEI_ENVIADO de -50.

Primer envío: HTTP 201

```
{
  "id": 1,
  "idempotencyKey": "veraza-amy-s05-001",
  "claveOrigen": "002180000000000001",
  "bancoDestino": "BBVA",
  "claveDestino": "0121800000000000099",
  "monto": 50,
  "estado": "ENVIADO"
}
```

Reintento: HTTP 201

```
{
  "id": 1,
  "idempotencyKey": "veraza-amy-s05-001",
  "claveOrigen": "002180000000000001",
  "bancoDestino": "BBVA",
  "claveDestino": "0121800000000000099",
  "monto": 50.0,
  "estado": "ENVIADO"
}
```



```
Pruebas HTTP — SPEI e idempotencia

$ POST /spei Idempotency-Key: veraza-amy-s05-001 (primer envío)
HTTP 201 {"id": 1, "idempotencyKey": "veraza-amy-s05-001", "claveOrigen":
"002180000000000001", "bancoDestino": "BBVA", "claveDestino": "0121800000000000099",
"monto": 50, "estado": "ENVIADO"}

$ POST /spei Idempotency-Key: veraza-amy-s05-001 (reintento)
HTTP 201 {"id": 1, "idempotencyKey": "veraza-amy-s05-001", "claveOrigen":
"002180000000000001", "bancoDestino": "BBVA", "claveDestino": "0121800000000000099",
"monto": 50.0, "estado": "ENVIADO"}

$ GET /cuentas/002180000000000001
HTTP 200 saldo: 750.0 (un solo descuento de 50)
```

Captura 3. Los dos envíos regresan el mismo SPEI y el saldo queda en 750.

La tabla de solicitudes SPEI impone unicidad sobre la clave. El servicio reserva la clave antes de cobrar; si ya existe una solicitud terminada, devuelve la misma fila sin ejecutar de nuevo el cargo. Las notificaciones y movimientos confirman un solo cobro.

4. Parada de app y persistencia

Detuve app con docker compose stop app. Tras 15 segundos no volvió sola y la solicitud HTTP falló. La base siguió healthy. Al iniciar app otra vez, la cuenta conservó saldo 750. También sobrevivió a docker compose down y docker compose up -d. Después de docker compose down -v y un nuevo arranque, consultar esa CLABE devolvió 404: se eliminó el volumen con la base.

app detenida → conexión rechazada
stop/start → saldo 750
down/up → saldo 750
down -v/up → GET /cuentas/002180000000000001: HTTP 404

```
Docker Compose — parada y volumen

$ docker compose stop app
Container mexi-banco-app-1 Stopped
$ después de 15 segundos: curl /actuador/health
Conexión rechazada; app no se reinició sola. db continuó healthy.
$ docker compose start app; GET cuenta origen
HTTP 200 saldo: 750.0
$ docker compose down; docker compose up -d; GET cuenta origen
HTTP 200 saldo: 750.0 (el volumen conservó los datos)
$ docker compose down -v; docker compose up -d; GET cuenta origen
HTTP 404 No existe la CLABE (el volumen fue eliminado)
```

Captura 4. Prueba de parada, persistencia y eliminación del volumen.

5. Código y configuración

El monolito tiene módulos cuenta, movimiento, transferencia, notificación y SPEI. Cada módulo separa controlador HTTP, servicio con reglas de negocio, repositorio de datos y entidad. El archivo Compose define app y db en la red mexibanco; app resuelve db por nombre y espera su healthcheck. El volumen nombrado mexibanco_datos guarda PostgreSQL. El Dockerfile usa JDK 21 para compilar y JRE 21 para ejecutar el JAR, reduciendo la imagen de ejecución.

6. Errores y conclusiones

Caso	HTTP
CLABE inexistente	404
CLABE repetida	409
sin CLABE	400
saldo insuficiente	422
misma cuenta	422
SPEI sin clave	400

Después de estos errores, el saldo de origen permaneció en 1000 en la base nueva.

¿Qué aportó Compose? Levantó y conectó los dos servicios, esperó la salud de db y conservó los datos mediante el volumen. Me sorprendió que detener app manualmente no la reinició.

¿Qué resolvería restart: always? Reiniciaría el contenedor en este mismo host ante una caída o parada no intencional. No reubica la carga si falla la máquina, ni proporciona réplicas, balanceo, descubrimiento o actualizaciones coordinadas. Además, una parada manual puede quedar respetada hasta que se reinicie el daemon o el contenedor.

¿Qué falta al escalar app? Cada réplica necesita un puerto distinto del host; sin balanceador, las peticiones al 8080 llegan solo a una. Un Service de Kubernetes ofrece una dirección estable y distribuye tráfico entre réplicas sanas.